Jerome-Reconfiguration:

```
#include<bits/stdc++.h>
using namespace std;

int main()
{
    int x, y;
    cin >> x >> y;

    int M = x;
    int num = 1;
    int mod = M % y;
    while(mod)
    {
        num = num * 10;
        M = M * 10;

        int val = (y - M % y);
        if(val < num)
        {
            M += val;
        }
        mod = M % y;
    }
    cout << M;
    return 0;
}
```

Harry - Lottery:

```
#include<bits/stdc++.h>
using namespace std;

vector<bool> Prime;
void sive()
{
```

```
    Prime[0]= Prime[1] = false;
    for(int i = 2; i * i <= Prime.size(); i++)
    {
        if(Prime[i] == false)
            continue;

        for(int j = i ; j * i < Prime.size(); j++)
        {
            if(Prime[j * i])
                Prime[j * i] = false;
        }
    }
}

int main()
{
    int n, k;
    cin >> n >> k;
    Prime.resize(1e5 + 20, true);
    sive();

    int sum = 2 + 5;

    while(n)
    {
        if(Prime[k])
        {
            sum += k;
            n--;
        }
        k += 10;
    }
    cout<<sum<<endl;

}
```

YETI:

```cpp
#include<bits/stdc++.h>
using namespace std;

int main()
{
    string str;
    cin >> str;

    long long sum = 0;
    string cur;
    string part1;
    for(int i = 0; i < str.size(); i++)
    {
        if(str[i] >='0' & str[i] <= '9')
            cur.push_back(str[i]);
        else{
            if(cur.size() != 0)
                sum += stoll(cur);
            part1.push_back(str[i]);
            cur = "";
        }
    }
    if(cur.size() != 0)
        sum += stoll(cur);
    cout<<part1<<endl;
    cout<<sum<<endl;
}
```

Pattern:

```cpp
#include<iostream>
using namespace std;

string helper(string a, string b, int c){

    string res = "";
    res += a;
```

```
    for(int i=0; i<stoi(a); i++){
        res += b;
    }

    for(int i=0; i<(stoi(b)+c); i++){
        res += a ;
        for(int i=0; i<(stoi(b)); i++){
            res += b;
        }
    }
    return res;
}
int main(){
    int a,b,c;
    cout<<"Enter a b c: ";
    cin>>a>>b>>c;
    //a = 2,b=3,c=3;
    string ans = "";

    for(int i=0; i<c; i++){
        ans += helper(to_string(a), to_string(b), i);
    }

    ans += to_string(a);

    for(int i=0; i<a; i++){
        ans += to_string(b);
    }

    ans += to_string(a);

    cout<<ans;

    return 0;
}
```

Tree:

```
#include<bits/stdc++.h>
using namespace std;

class node{
    public:
        int val;
        node *left, *right;

        node(int x)
        {
            val = x;
            left = right = NULL;
        }
};

bool balanced;
int sum;
void postorder(node* r)
{
    if(r == NULL)
        return;


    if(r->left && r->left->val > r->val)
        balanced = false;

    if(r->right && r->right->val > r->val)
        balanced = false;

    if(!r->left && !r->right)
        sum += r->val;

    postorder(r->left);
    postorder(r->right);
}

void builTree(node* root, string str, int val)
```

```
{

    node* r = root;
    int n = str.size();

    for(int i = 0; i < n - 1; i++)
    {
        if(str[i] == 'L')
            r = r->left;
        else
            r = r->right;
    }
    node* temp = new node(val);
    if(str[n - 1] == 'L')
        r->left = temp;
    else
        r->right = temp;
}

int main()
{
    int n, root_val;
    cin >> n >> root_val;
    vector<pair<string, int>> List;
    int val;
    string str;
    for(int i = 0; i < n - 1; i++)
    {
        cin >> str >> val;
        List.push_back({str, val});
    }

    sort(List.begin(), List.end());

    node* root = new node(root_val);

    balanced = true;
    sum = 0;
```

```
    for(int i = 0; i < List.size() ;i++)
        builTree(root, List[i].first, List[i].second);



    if(balanced)
    {
        cout<<abs(root_val - sum);
    }else{
        cout<<root_val + sum;
    }
    return 0;
}
```

Difference min max:

```
#include<bits/stdc++.h>
using namespace std;

int main()
{
    int n;
    cin >> n;
    vector<vector<int>> mat(n, vector<int>(n));

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            cin >> mat[i][j];
        }
    }
    int MIN = 0;
    int MAX = 0;

    for(int i = 0; i < n; i++)
    {
```

```
        int min_val = INT_MAX, max_val = INT_MIN;
        for(int j = 0; j < n; j++)
        {
            if(mat[i][j] < min_val)
                min_val = mat[i][j];

            if(mat[i][j] > max_val)
                max_val = mat[i][j];
        }
        MIN += min_val;
        MAX += max_val;
    }
    cout<<MAX - MIN;
    return 0;
}
```

Encryption- Spiral Matrix:

```
#include<bits/stdc++.h>
using namespace std;

void _rotate(vector<int>& arr, int k)
{
    reverse(arr.begin(), arr.begin() + k);
    reverse(arr.begin() + k, arr.end());
    reverse(arr.begin(), arr.end());
}

void encrypt(vector<int>& vec,int m,int n,int r)
{
    vector<vector<int>> mat(m, vector<int>(n));

    int idx = 0;

    for(int i = 0; i < m; i++)
    {
        for(int j = 0; j < n; j++)
```

```
        {
            mat[i][j] = vec[idx++];
        }
    }


    int top = 0, bottom = m - 1, left = 0, right =  n - 1;

    while(top <= bottom && left <= right)
    {

        if(bottom - top >= 1 && right - left >= 1)
        {

            vector<int> temp;

            for(int i = left ; i <= right; i++)
                temp.push_back(mat[top][i]);

            for(int i = top + 1; i <= bottom; i++)
                temp.push_back(mat[i][right]);


            for(int i = right - 1; i >= left; i--)
                temp.push_back(mat[bottom][i]);


            for(int i = bottom - 1; i > top; i--)
                temp.push_back(mat[i][left]);


            _rotate(temp, r);

            int idx = 0;


            for(int i = left ; i <= right; i++)
                mat[top][i] = temp[idx++];
```

```
            for(int i = top + 1; i <= bottom; i++)
                mat[i][right] = temp[idx++];

            for(int i = right - 1; i >= left; i--)
                mat[bottom][i] = temp[idx++];

            for(int i = bottom - 1; i > top; i--)
                mat[i][left] = temp[idx++];

        }

        top++;
        right--;
        bottom--;
        left++;
    }

    for(int i = 0; i < m; i++)
    {
        for(int j = 0;j < n; j++)
        {
            cout<<mat[i][j]<<" ";
        }
        cout<<endl;
    }

}

int main()
{
    int x;
    cin >> x;
    vector<int> vec(x);

    for(int i = 0; i < x; i++)
        cin >> vec[i];

    int m,n,r;
    cin >> m >> n >> r;
```

```
    encrypt(vec, m, n, r);

    return 0;
}
```

BOB - Calculator:

```
#include<bits/stdc++.h>
using namespace std;

int precedence(char op)
{
    if(op == '+' || op == '-')
        return 1;
    return 2;
}

int operation(int val1, int val2, char op)
{
    if(op == '+')
        return val1 + val2;
    else if(op == '-')
        return val1 - val2;
    else if(op == '*')
        return val1 * val2;
    else
        return val1 / val2;
}

int evalute(string& exp)
{
    vector<int> operand_s;
    vector<char> operator_s;

    for(int i = 0; i < exp.size();)
```

```
    {
        if(exp[i] <= '9' && exp[i] >= '0')
        {
            string num = "";
            while( i < exp.size() && exp[i] <= '9' && exp[i] >= '0' )
            {
                num += exp[i];
                i++;
            }
            operand_s.push_back(stoi(num));
        }
        else if(exp[i] == '(')
        {
            operator_s.push_back(exp[i]);
            i++;
        }
        else if(exp[i] == ')')
        {
            while(operator_s.size() > 0 && operator_s.back() != '(')
            {
                // for(int i = 0; i < operand_s.size(); i++)
                //  cout<<operand_s[i]<<" ";
                // cout<<endl;
                int val2 = operand_s.back();
                operand_s.pop_back();
                int val1 = operand_s.back();
                operand_s.pop_back();
                char op = operator_s.back();
                operator_s.pop_back();
                int val = operation(val1, val2, op);
                operand_s.push_back(val);
            }
            operator_s.pop_back();
            i++;
        }
        else{

            while(operator_s.size() && operator_s.back() != '(' &&
precedence(operator_s.back()) >= precedence(exp[i]))
```

```
            {
                int val2 = operand_s.back();
                operand_s.pop_back();
                int val1 = operand_s.back();
                operand_s.pop_back();
                char op = operator_s.back();
                operator_s.pop_back();
                int val = operation(val1, val2, op);

                operand_s.push_back(val);
            }

            operator_s.push_back(exp[i]);
            i++;
        }
    }

    while(operator_s.size()){
        int val2 = operand_s.back();
        operand_s.pop_back();
        int val1 = operand_s.back();
        operand_s.pop_back();
        char op = operator_s.back();
        operator_s.pop_back();
        int val = operation(val1, val2, op);
        operand_s.push_back(val);
    }
    return operand_s.back();
}

string _replace(string exp, int val)
{
    for(int i = 0; i < exp.size(); i++)
    {
        if(exp[i] == 'x')
            exp[i] = '0' + val;
    }
    return exp;
}
```

```cpp
int main()
{
    int p, m;
    string exp;
    cin >> exp;
    cin >> p >> m;

    string exp0 = _replace(exp, 0);
    string exp1 = _replace(exp, 1);

    int val1 = evalute(exp0);
    // cout<<"val1 = "<<val1<<endl;
    int val2 = evalute(exp1);
    // cout<<"val2 = "<<val2<<endl;
    int d = val2 - val1;

    if(val1 % m == p)
    {
        cout << 0;
        return 0;
    }


    int i = 1;
    while(val2 % m != p)
    {
        val2 += d;
        i++;
    }
    cout<< i;
    return 0;
}
```